设计思路

选择协议

- TCP：可靠数据传输。适合 **寄存** 形式的文件。语音留言，视频留言
- UDP: 不可靠数据传输。适合**实时**的音视频传输。

项目结构

1. 用户前端代码文件：
2. 用户后端主文件：

   调用数据库文件获取好友、选择好友、选择语音文件（现在录取语音）、文件发送
3. 用户数据库文件：一个表格
4. 服务器后端主文件：维护
5. 服务器数据库文件（模拟）：一个表格（用户ID）
6.
7. 其他函数： 文件选择、文件处理、文件传输、文件接收、文件展示/播放
   - 音频获取函数：
     - 音频处理：音频获取
   - 文件传输函数：分为两个函数
     - 基于TCP的传输函数：语音留言时使用
       1. 输入：传输文件、目的地址、目的端口、源地址、源端口
       2. 压缩
     - 基于UDP的传输函数：语音通话时使用
       1. 输入：传输文件、目的地址、源地址
       2. 压缩
   - 文件接收：
   - 文件展示：

广义实现逻辑

- 服务器：
  - 数据库服务器：存储用户ID、IP地址
  - 网络服务器：在IP变动的情况下如何进行文件传输？只考虑离线情况下的IP变动
    - 语音留言： 邮箱工作原理（不确定接收端的IP地址，先缓存到服务器，接收端登录后进行发送）
    - 语音通话：在用户都在线的情况下才能发起

- 客户端：

- 电话本 / 通讯录（实际是维护一个本地的数据库）
- 客户端前端：UI
  - 登录页面：输入自己的userID
  - 聊天服务页面：
    1. 一对一传输：从电话本上选择对方的userID

       语音留言：

       语音通话
    2. 会议室（IP组播）：创建会议室、加入会议室、终止会议室

       仅语音留言： -> 微信群聊

       语音通话：腾讯会议

项目文件拆分

设计思路流程图

以下是基于您提供的设计思路绘制的系统流程图，展示了客户端与服务器的主要交互流程，包括登录、一对一语音留言/通话以及会议室功能。

项目编写步骤指导

按照以下步骤，您可以从零开始逐步构建这个语音通信系统。假设使用 Python 作为开发语言，利用其丰富的网络和音频库。

1. 项目规划与准备

- **创建项目目录**，并在其中建立以下子目录：
  - `client/`：客户端代码
  - `server/`：服务器代码
  - `common/`：公共模块（如协议定义、压缩函数）
  - `data/`：存放离线留言文件、本地数据库等

2. 设计通信协议

定义客户端与服务器之间的消息格式（可采用简单的 JSON 或自定义二进制头）。例如：

- **登录请求**：`{"type":"login", "user_id":"Alice"}`
- **登录响应**：`{"type":"login_resp", "status":"ok", "friends_online":["Bob"]}`
- **在线查询**：`{"type":"query_online", "target_id":"Bob"}`
- **在线响应**：`{"type":"query_resp", "online":true}`
- **离线留言上传**：先发送文件头，再发文件内容，可自定义格式。

3. 实现公共模块（`common/`）

- `network.py`：封装 TCP 和 UDP 的发送/接收函数。
  - `tcp_send_file(file_path, addr, port)`：将文件通过 TCP 发送到指定地址。
  - `tcp_recv_file(save_path, conn)`：接收 TCP 文件。
  - `udp_send_audio(data, addr, port)`：通过 UDP 发送音频数据块。
  - `udp_recv_audio(sock)`：接收 UDP 音频数据。
- `audio.py`：音频录制与播放（使用 `pyaudio`）。
  - `record_audio(duration, save_path)`：录制指定时长音频并保存为文件。
  - `play_audio(file_path)`：播放音频文件。
  - `stream_audio(callback)`：实时采集/播放音频流，用于通话。

4. 实现客户端本地数据库

- 使用 SQLite 或简单的 JSON 文件存储好友信息。例如 `phonebook.json`：

```
{
  "my_id": "Alice",
  "friends": [
    {"id": "Bob", "alias": "Bob", "last_online": "..."}
  ]
}
```

- 编写 `phonebook.py` 提供增删改查接口。

5. 实现服务器端核心逻辑

- `server.py`：
  - 维护在线用户字典：`online_users = {user_id: (ip, port)}`
  - 维护离线留言队列：`offline_msgs = {user_id: [file_paths]}`
  - 实现 TCP 服务器线程，处理客户端请求：
    - 登录：记录 IP 和端口，返回在线好友列表。
    - 在线查询：返回指定用户是否在线。
    - 离线留言上传：保存文件到 `data/offline/`，并记录到离线队列。
    - 用户下线：从在线字典中移除。
  - 当用户上线时，检查离线留言队列，主动推送留言文件。

6. 实现客户端后端逻辑

- `client_backend.py`：
  - 封装与服务器的通信函数：
    - `login(user_id)`：连接服务器并登录。
    - `query_online(friend_id)`：查询好友在线状态。

        - `upload_offline_msg(target_id, file_path)`：上传离线留言。
    - 实现语音留言流程：
        - 调用 `audio.record_audio()` 录制。
        - 根据好友在线状态，直接发送或上传至服务器。
    - 实现语音通话流程：
        - 检查好友在线。
        - 若在线，启动 UDP 线程实时采集、发送、接收音频。

7. 实现客户端前端 UI

- 使用 `tkinter` 构建图形界面：
    - **登录窗口**：输入 userID，点击登录。
    - **主窗口**：显示好友列表（带在线状态），提供“语音留言”、“语音通话”、“会议室”按钮。
    - **录音窗口**：弹出窗口，包含“开始录音”、“停止”、“发送”按钮。
    - **通话窗口**：显示通话状态，包含“挂断”按钮。
    - **会议室窗口**：输入会议室地址或创建新会议室。

8. 实现会议室功能

- **IP 组播**：使用 UDP 组播实现多人语音。
- 在客户端：
    - 创建会议室：选择一个组播地址（如 `224.1.1.1`）和端口，加入组播组，并通知服务器（可选）。
    - 加入会议室：输入组播地址，加入组播组。
    - 语音通信：所有成员通过组播地址发送和接收音频数据。
- 服务器可辅助管理会议室列表，但组播通信本身不经过服务器。